

Documentación del Proyecto / Atributos de Ingeniería

Proyecto: Defensa y Asalto de Base

Integrantes: Evan Alejandro Umaña Sojo y Yosep Díaz Marín

1. Atributo de Análisis de Problema

1.1 Problema complejo de ingeniería

El proyecto aborda el diseño e implementación de un sistema de software interactivo “un juego de estrategia por turnos” que debe coordinar múltiples subsistemas (lógica de combate, economía, pathfinding, persistencia y audio) manteniendo una interfaz responsiva y comportamiento justo para ambos jugadores. Requiere aplicar matemáticas discretas (geometría de cuadrículas, probabilidad, modelos económicos), ciencias de la computación (estructuras de datos, complejidad algorítmica, búsqueda de caminos) e ingeniería de software (arquitectura modular, diseño orientado a objetos), sin una solución única evidente. En cuanto a sostenibilidad, al ser una aplicación de escritorio liviana sin dependencia de servidores, su huella energética es mínima y puede ejecutarse en equipos de gama baja, extendiendo su vida útil y fomentando la interacción social presencial como alternativa accesible a videojuegos comerciales.

1.2 Análisis del contexto y variables

El sistema corre en equipos personales en un contexto educativo con recursos modestos. Las variables clave son: cantidad de objetos simultáneos en el tablero (afecta el costo computacional de rangos y colisiones), flujo de dinero entre rondas (debe balancearse para evitar ventajas estructurales), y la concurrencia entre Tkinter y pygame, que deben sincronizarse sin bloquear la interfaz. Desde la óptica de sostenibilidad, se consideró la eficiencia algorítmica como factor de consumo energético: algoritmos de mayor complejidad innecesaria implican mayor uso de CPU y electricidad.

1.3 Plan de solución

La solución separa la lógica del juego (game.py), la interfaz (game_canvas.py), la orquestación (main.py) y el audio (config.py). Para la detección eficiente de colisiones y rangos se usan diccionarios indexados por posición ($O(1)$), evitando recorridos lineales. El movimiento de unidades se resuelve con el algoritmo A* sobre una cola de prioridad (heapq). La persistencia de jugadores usa archivos JSON, sin infraestructura de servidor. La música se maneja con un sondeo no bloqueante (root.after) para no interferir con el ciclo de eventos de Tkinter.

1.4 Ventajas y desventajas

Ventajas:

- Técnicas: arquitectura modular facilita agregar torres, unidades o facciones sin reescribir el núcleo.
- Ambientales/económicas: A* y acceso $O(1)$ reducen consumo de CPU; JSON elimina el costo de un servidor.
- Sociales: accesible sin internet ni hardware de alta gama.

Desventajas:

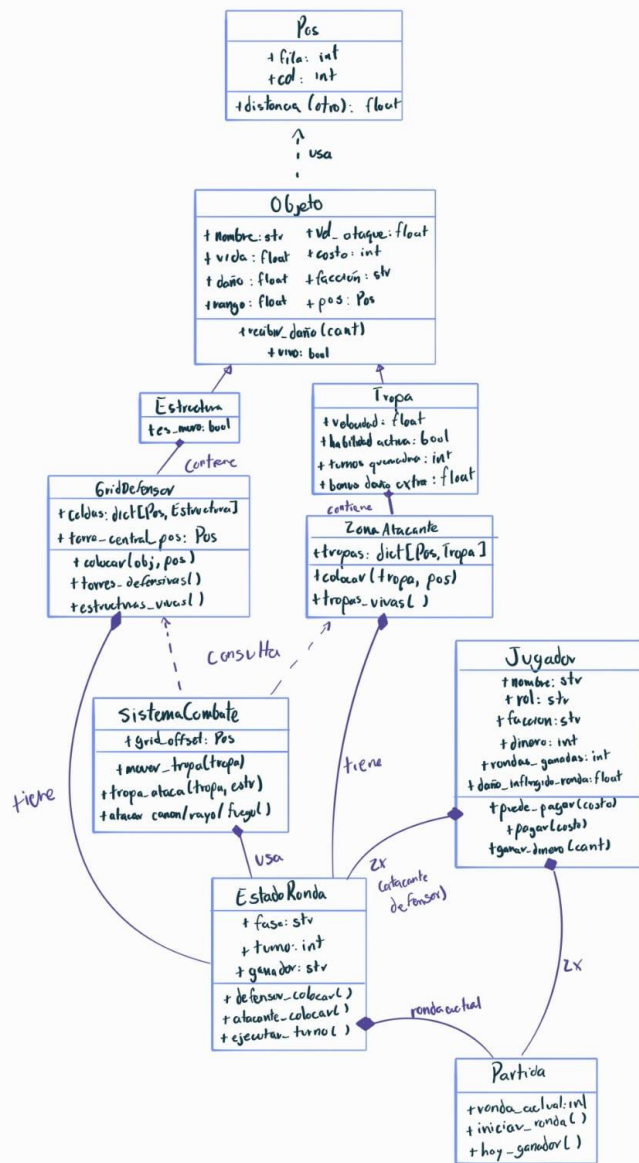
- JSON no escala a partidas remotas o múltiples usuarios concurrentes (riesgo de condiciones de carrera).
- Tkinter de un solo hilo puede ser cuello de botella si el tablero creciera significativamente.
- Al ser local, no permite juego remoto, aunque evita el costo ambiental de mantener servidores.

2. Atributo de Herramientas de Ingeniería

2.1 Técnicas, recursos y herramientas utilizadas

Se usó Python 3 con Tkinter (interfaz gráfica) y pygame (audio). El diseño aplica programación orientada a objetos con jerarquía de herencia (Objeto → Estructura / Tropa) y composición (GridDefensor y ZonaAtacante contienen los objetos; EstadoRonda coordina el combate; Partida gestiona jugadores y rondas). Para el pathfinding se implementó A* con heapq. El control de versiones se gestionó con Git/GitHub, y la depuración mediante scripts de prueba aislados que ejecutan la lógica sin interfaz gráfica.

2.2 Diagrama de clases (modelo de objetos)



2.3 Aplicación durante el desarrollo

La jerarquía de clases se aplicó mediante funciones fábrica (`crear_torre_fuego()`, `crear_tanque()`) que instancian `Estructura` o `Tropa` con sus parámetros, evitando duplicar lógica de vida y daño. A* se usa en `SistemaCombate` para recalculando rutas por turno. Los diccionarios en `GridDefensor` y `ZonaAtacante` permiten validar colocaciones y detectar colisiones sin iterar todos los objetos. El patrón `root.after()` en `config.py` alterna canciones de la playlist sin bloquear Tkinter.

2.4 Adaptación y refinamiento

Durante las pruebas se detectaron problemas que llevaron a refinar las herramientas: el cálculo de rangos de ataque fallaba por mezclar coordenadas locales del grid con coordenadas globales, corregido añadiendo una capa de conversión (`grid_offset`) en `SistemaCombate`. El retorno al menú principal duplicaba la ventana al reimportar el módulo; se reemplazó por inyección de dependencias mediante un callback. El modelo económico se rediseñó de reinicio total a conservación con bonificaciones calculadas. El balance de audio se ajustó iterativamente para que música y efectos fueran audibles simultáneamente.